Software Design Description (SDD)

1. Purpose of the Document

The purpose of this Software Design Description (SDD) is to provide a comprehensive architectural and logical overview of the **SentinelsVault** system. It translates the functional requirements into a modular, object-oriented design. This document serves as the primary reference for developers and security auditors to understand the system's internal structure, data flow, and cryptographic implementation.

1.1 System Overview

SentinelsVault is an enterprise-grade, local-first cryptographic password manager. Unlike cloud-based solutions, SentinelsVault operates on a **Zero-Knowledge Architecture**, ensuring that the master key never leaves the user's local RAM. The system provides secure storage via **AES-256** encryption, identity verification through **Multi-Factor Authentication (MFA)**, and proactive risk management via an integrated **Security Auditor**

1.2 Design Goals and Principles

Security First: Implementation of industry-standard protocols (Argon2/PBKDF2 and AES-256-GCM).

Modular Design: Using Object-Oriented Programming (OOP) to separate the GUI, Database, and Cryptography logic.

Data Integrity: Ensuring ACID properties (Atomicity, Consistency, Isolation, Durability) through SQLite.

Offline Resilience: Minimizing the attack surface by eliminating remote server dependencies."

1.3 Scope

1. The **Encryption Engine** for data at rest.
2. The **Key Derivation Function (KDF)** for secure master password handling.
3. The **Relational Database Schema** for password metadata.
4. The **Security Audit Logic** for password strength and reuse analysis.

Note: This design excludes cloud-syncing to maintain the air-gapped security model."

Term	Definition
AES-256	Advanced Encryption Standard with a 256-bit key (Military Grade).
KDF	Key Derivation Function; turns a password into a cryptographic key.
Zero-Knowledge	A design where the service provider (or software) has no access to user keys.
Salt	Random data added to a password before hashing to prevent rainbow table attacks.

2. System Architecture

The first part of your SDD shows how the different "blocks" of your software talk to each other. For **SentinelsVault,** a **Layered Architecture** is used.

The system has three layers:

Presentation Layer: The GUI (Tkinter) where the user clicks buttons.

Application Logic Layer: The "Brain" (The Python classes that handle encryption and auditing).

Data Layer: The SQLite database.

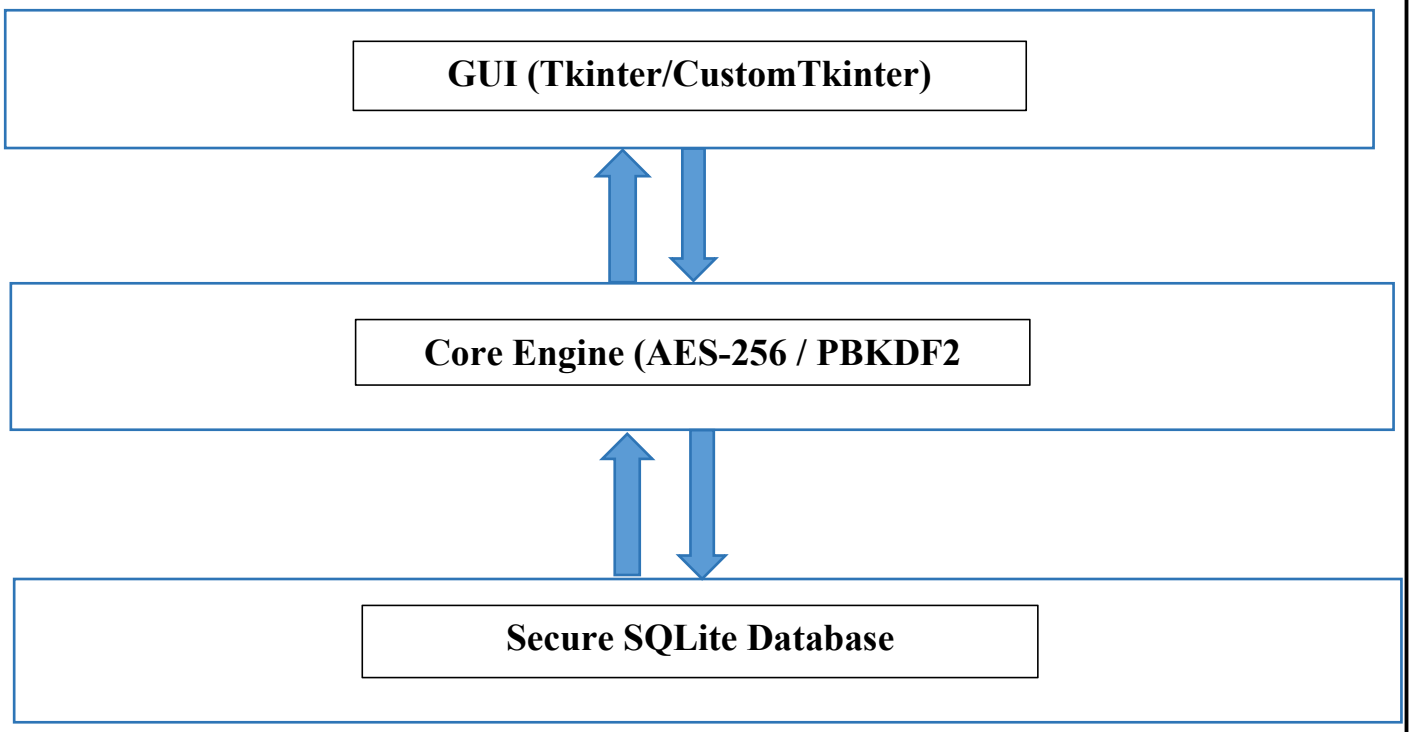


Figure :- Three Layered System Architecture

3. Class Diagram

Since OOP (Object-Oriented Programming) is used here, therefore classes are show

DatabaseManager: Handles SQL queries (Add, Delete, Fetch).

CryptoEngine: Handles encrypt() and decrypt() functions.

AuthManager: Handles Master Password hashing and session login.

SecurityAuditor: Scans for weak passwords.

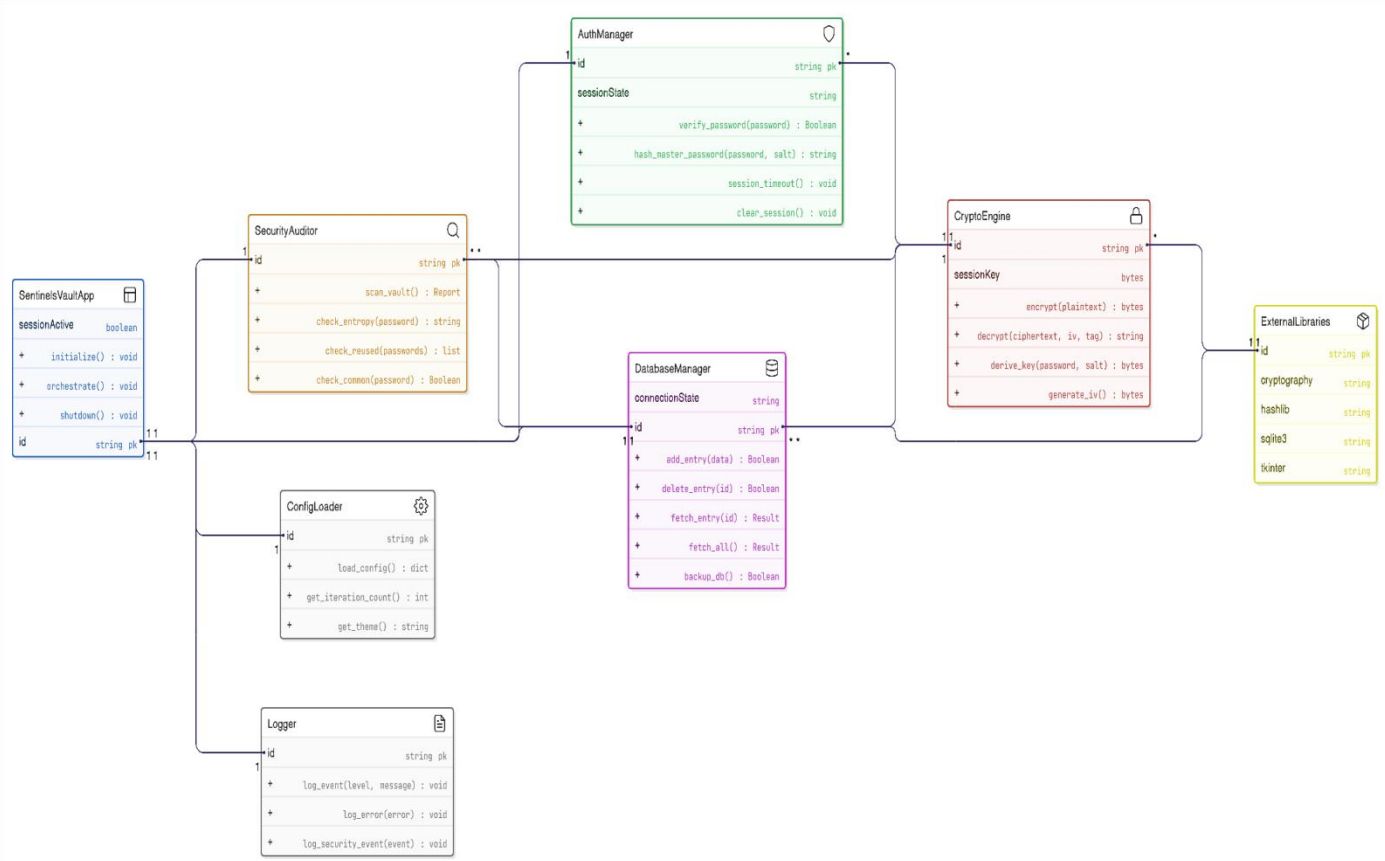


Figure :- Class Diagram for SentinelsVault

4. Modular Design Overview: The system is designed using **Encapsulation**. Each class has a specific, isolated responsibility:

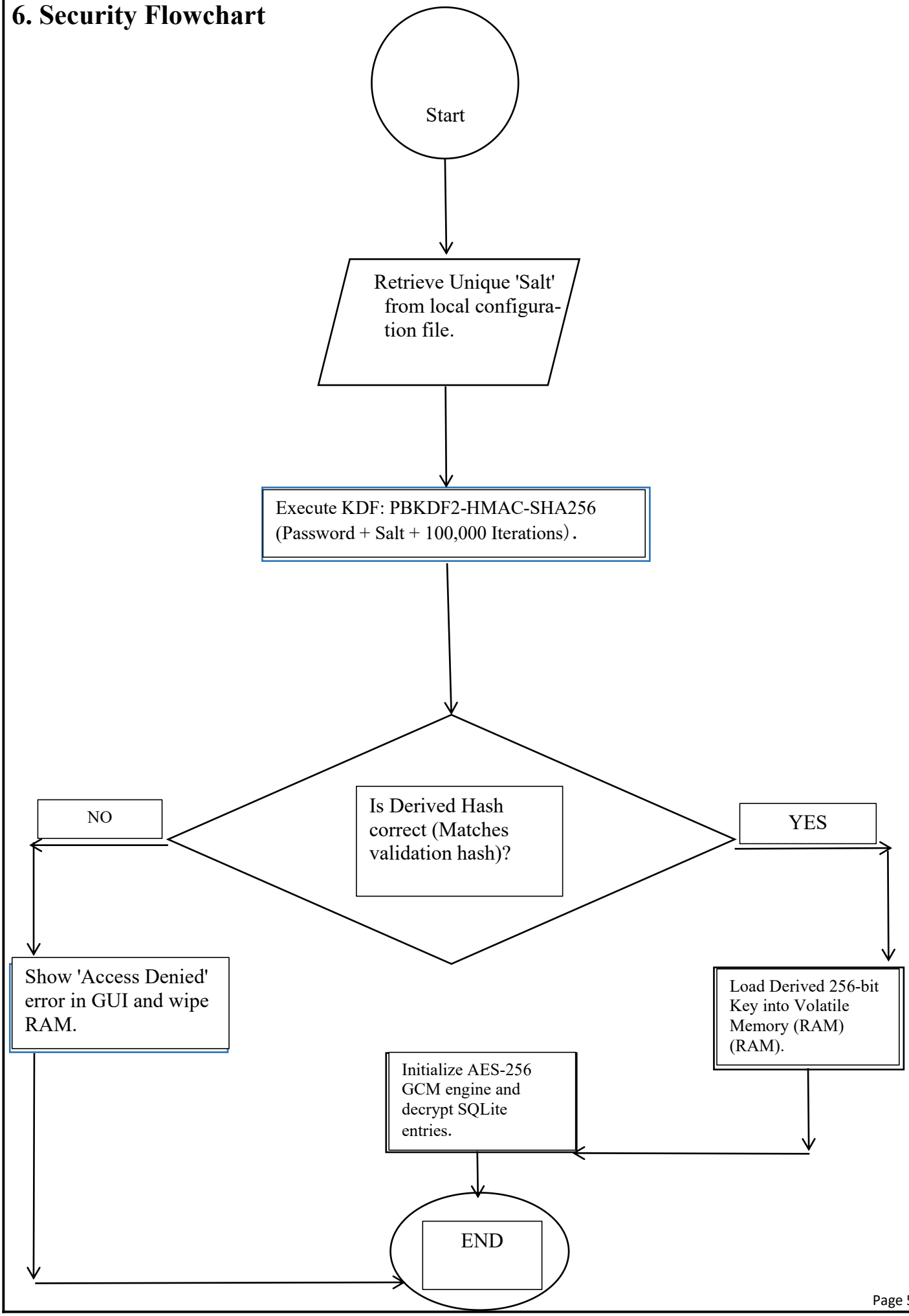
- **AuthManager**: Acts as the gatekeeper. It ensures no user can access the vault without the correct Master Password hash comparison.
- **CryptoEngine**: The cryptographic heart. It handles the **AES-256 GCM** implementation. It is the only class that touches the raw encryption keys.
- **DatabaseManager**: A dedicated wrapper for SQLite. It ensures all database transactions are **ACID-compliant**.
- **SecurityAuditor**: A standalone logic module that analyzes stored passwords against entropy rules and common password lists without compromising the encryption.

5. Database Schema

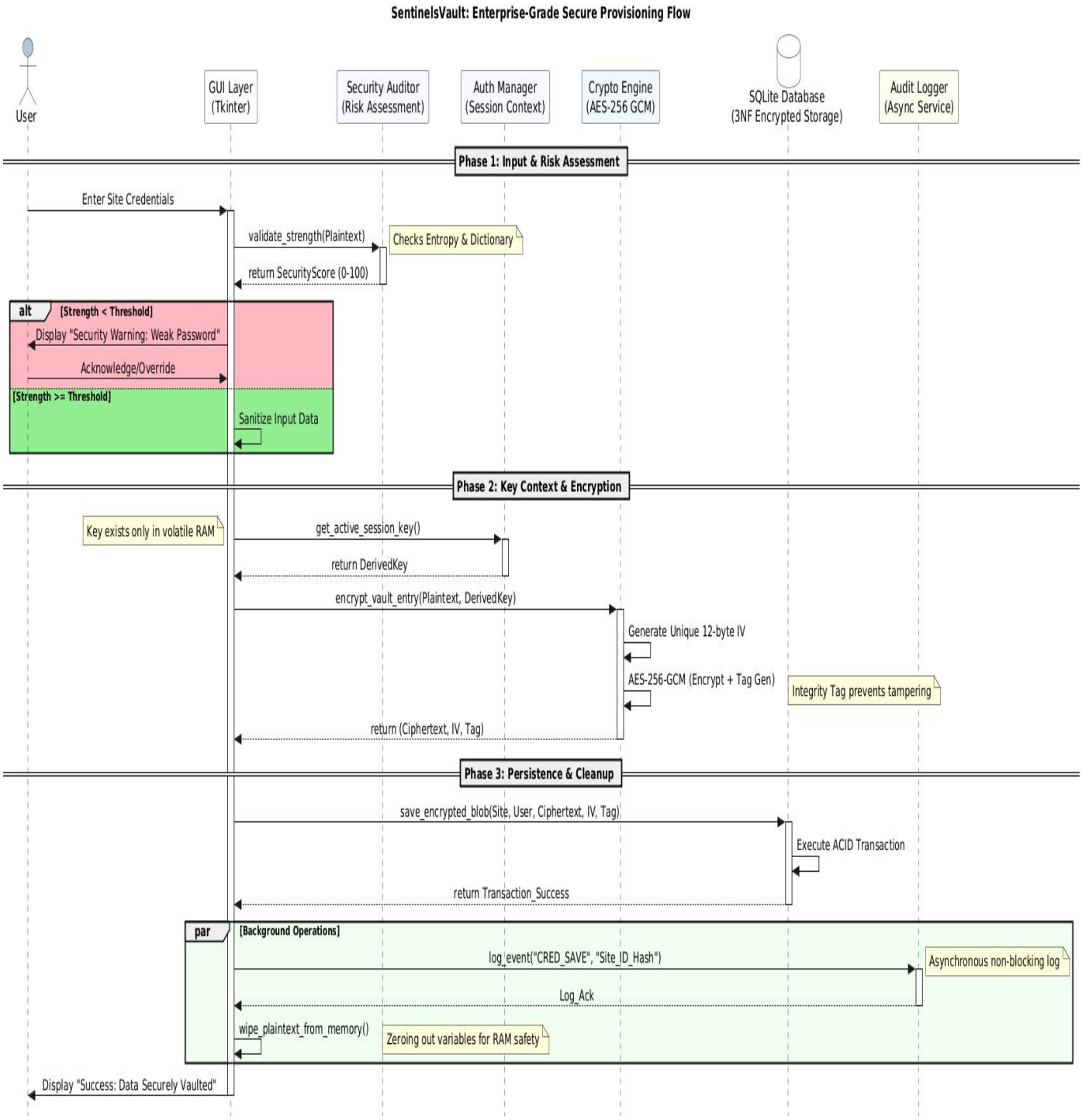
SentinelsVault: Database Schema (vault_entries): - This table represents the "at-rest" state of your data. To an outsider, everything in the encrypted_password is unreadable without the Master Password.

Column Name	Data Type	Constraint	Technical Purpose
id	INTEGER	PRIMARY KEY, AUTOINCREMENT	Unique identifier for each entry to ensure database normalization.
site_name	TEXT	NOT NULL	The name of the service (e.g., "KIIT Portal"). This is stored in plain text for easy searching.
encrypted_username	BLOB	NOT NULL	**Ciphertext Identity:** The email or ID is encrypted using AES-256. This prevents "Identity Leaking" if the database file is stolen.
encrypted_password	BLOB	NOT NULL	**Ciphertext Secret:** The actual password, scrambled using AES-256. This is the core protected data.
iv	BLOB	NOT NULL	**Initialization Vector:** A unique random number for this specific row. It ensures that even if two accounts have the same username or password, the BLOB data looks completely different.
salt	TEXT	NOT NULL	**Cryptographic Salt:** Used to derive the unique key for this entry, protecting against pre-computation attacks.

6. Security Flowchart



7. Sequence Diagram



The sequence logic ensures a **Secure Transaction Flow**. By utilizing a decoupled interaction model, the system ensures that the **Plaintext password** exists only momentarily in the GUI's local scope before being passed to the CryptoEngine. Once the Encrypt_data() method returns the ciphertext, the plaintext is immediately cleared from the GUI's memory buffer, and only the encrypted BLOB is transmitted to the Database Layer.

8. Low Level Design (LLD) Specifications

8.1 Cryptographic Standards

Encryption Algorithm: AES-256-GCM (Advanced Encryption Standard).

> *Reasoning:* Unlike older modes (like CBC), GCM provides **Authenticated Encryption**. It ensures that if an attacker modifies the encrypted file, the system will detect it and refuse to decrypt it.

Key Derivation: PBKDF2-HMAC-SHA256.

> *Reasoning:* We use **100,000 iterations** to turn the Master Password into a 256-bit key. This "Key Stretching" makes it virtually impossible for a hacker to "Brute-Force" the password using modern hardware.

8.2 Data Integrity & Database Design

- **ACID Compliance:** The system ensures that every database transaction is **Atomic** (all or nothing) and Consistent.

- **Database Normalization:** The database is designed in **3rd Normal Form (3NF)**.

  - *Primary Key:* ID (Unique identifier).
  - *Data Blobs:* Encrypted passwords and IVs are stored as **BLOB** (Binary Large Objects) to prevent encoding errors.

8.3 Memory Management & Session Security

- **Zero-Persistence Principle:** The encryption key derived from the Master Password is **never written to** the hard drive.

- **RAM Management:** Once the user logs out or the application window is closed, the variable holding the key is overwritten with null values or deleted using Python's garbage collector to prevent **"Memory Dumping"** attacks.

- **Local-First Architecture:** No data is transmitted over a network, removing the risk of "Man-in-the-Middle" (MITM) attacks.

8.4 Error Handling Strategy

The system implements a robust **Try-Except-Finally** architecture for all critical modules."

- **Try:** Attempt the database connection or decryption.

- **Except:** Handle specific errors (e.g., `InvalidToken` for wrong passwords or `sqlite3.Error` for database corruption).
- **Finally:** Ensure the **`database.close()`** command is always executed to prevent file locking and data corruption, even if the app crashes.

9. Entity Relationship Diagram

3rd Normal Form

- **1st Normal Form (1NF):** Each cell contains a single value (no lists of passwords in one row), and each record is unique via the `Entry_ID`.

- **2nd Normal Form (2NF):** All non-key attributes (Site_Name, IV, etc.) are fully functional and dependent on the Primary Key (`Entry_ID`).

- **3rd Normal Form (3NF):** There are no **transitive dependencies**. For example, the `IV` is only relevant to the `Encrypted_Pass` of that specific `Entry_ID`. No non-key column depends on another non-key column.

To maintain 3NF while maximizing security, the **Initialization Vector (IV)** is stored alongside the ciphertext. While the Master Key is derived once per session, a unique IV is generated for every row update. This ensures that if a user has the same password for 'Facebook' and 'Gmail,' the Encrypted_Pass values will look completely different in the database, preventing pattern-matching attacks.

9.1 The Entities (The Tables)

For a secure, multi-user capable (even if local) manager, we need two primary entities:

1. **Users Table:** Stores the "Master" credentials required to open the vault.
2. **VaultEntries Table:** Stores the actual encrypted accounts (Site, Username, Password).

9.2 Attributes and Constraints

Table A: Users (The Authentication Entity)

Attribute	Data Type	Constraint	Purpose
User_ID	INTEGER	**Primary Key**	Unique identifier for the vault owner.
Master_Hash	TEXT/BLOB	NOT NULL	The PBKDF2 hash of the Master Password.
Master_Salt	BLOB	NOT NULL	Unique salt used to hash the Master Password.
Iterations	INTEGER	NOT NULL	The number of PBKDF2 rounds (e.g., 100,000).
Created_At	DATETIME	DEFAULT NOW	Audit trail for account creation.

Table B: VaultEntries (The Credential Entity)

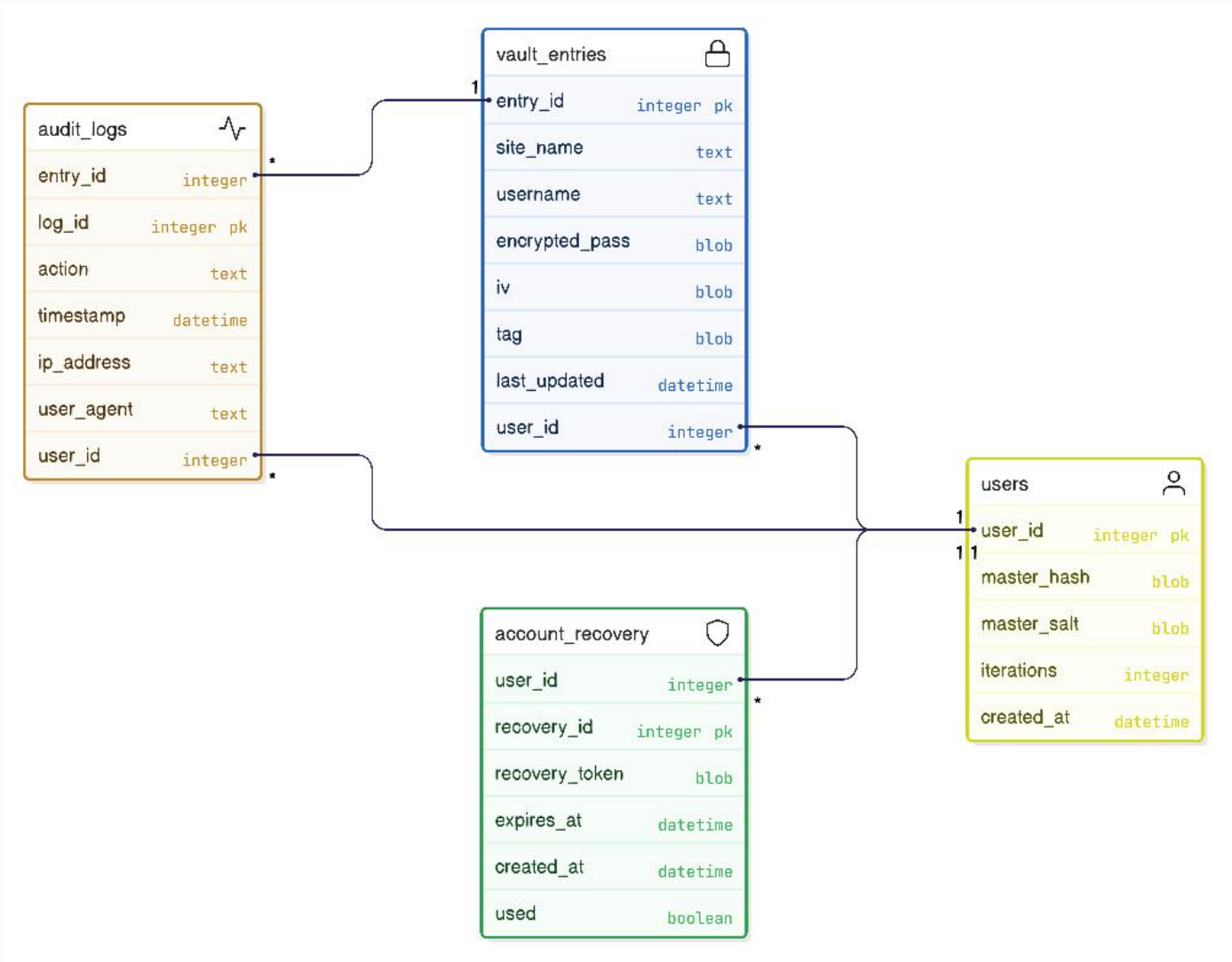
Attribute	Data Type	Constraint	Purpose
Entry_ID	INTEGER	**Primary Key**	Unique identifier for each saved login.
User_ID	INTEGER	**Foreign Key**	Connects the entry to the specific User.
Site_Name	TEXT	NOT NULL	The application/website name.
Username	TEXT	NOT NULL	The login ID for that site.
Encrypted_Pass	BLOB	NOT NULL	The AES-256 GCM ciphertext.

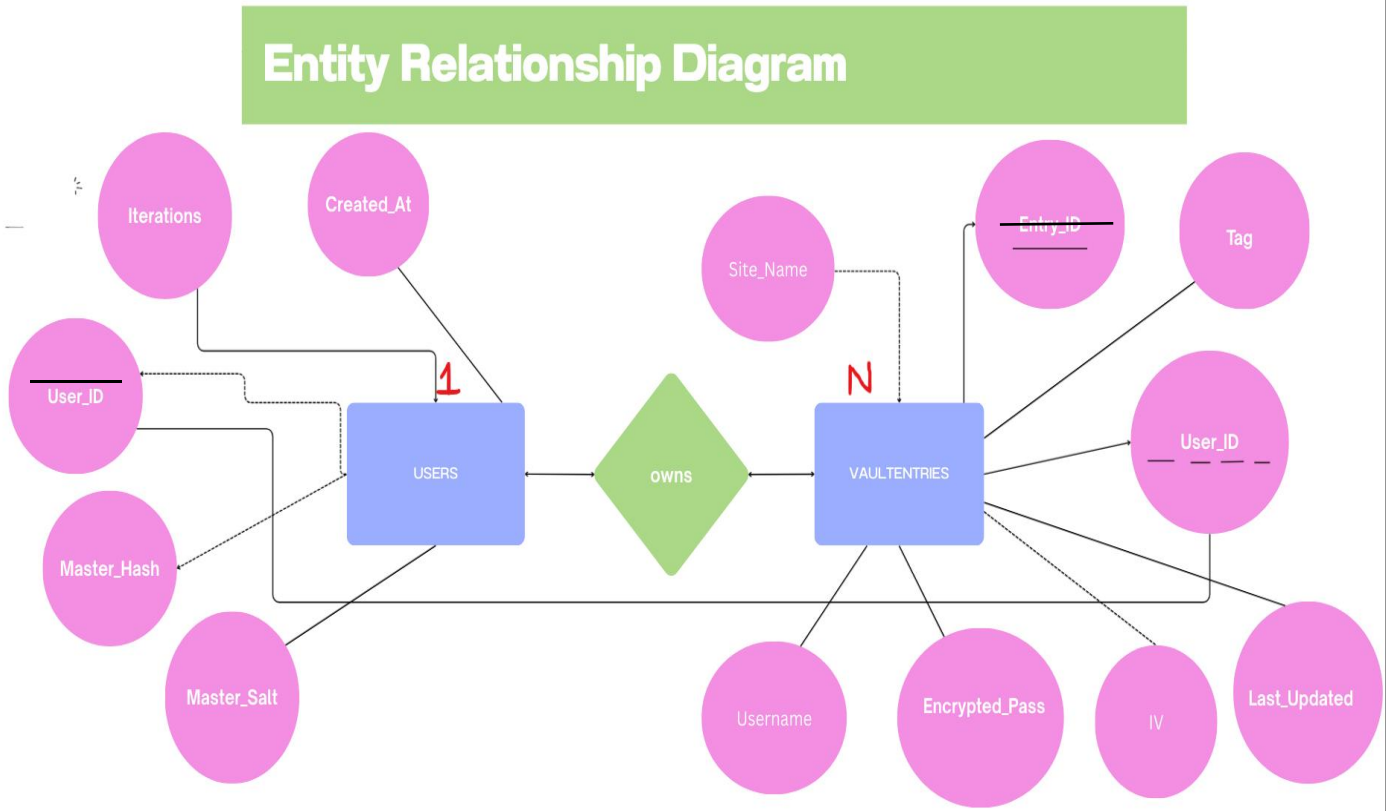
Attribute	Data Type	Constraint	Purpose
IV	BLOB	NOT NULL	**Initialization Vector** (unique for *every* row).
Tag	BLOB	NOT NULL	The GCM Authentication Tag (for integrity).
Last_Updated	DATETIME	DEFAULT NOW	Tracks when the password was last changed.

9.3 The Relationship:

This is a **One-to-Many (1:M)** relationship.

Logic: One **User** can have many **VaultEntries**, but one specific **VaultEntry** belongs to only one **User**.





9.4 Relationship Description (Cardinality): "The relationship between the **Users** entity and the **VaultEntries** entity is **One-to-Many (1:N)**.

- **Mandatory One (Users):** Each vault entry must be linked to exactly one authenticated User ID to ensure data ownership and isolation.
- **Optional Many (VaultEntries):** A single user can store zero or more encrypted password records.

This relationship is enforced via a **Foreign Key (`User_ID`)** in the `VaultEntries` table, which references the **Primary Key** in the `Users` table. This ensures **Referential Integrity**—if a user account is deleted, all associated vault entries are also removed (On Delete Cascade)."

10. Design Rationale

10.1 Key Relationships (How they rely on each other)

In your Class Diagram, the relationships are **Dependencies** and **Associations**.

- **The "Main" Dependency:** The `SentinelsVaultApp` (Main Class) acts as the **Orchestrator**. It "has-a" relationship with all other classes.
- **Security Dependency:** The `AuthManager` must return a "Success" state before the `DatabaseManager` is allowed to fetch any data.
- **Data Transformation:** The `DatabaseManager` relies on the `CryptoEngine` to decrypt the "BLOB" data it pulls from SQLite before that data can be sent to the GUI.

10.2 Most Important Aspects to Capture

For an "Enterprise-Grade" project, **Security Controls & Responsibilities** are the most important.

- **Encapsulation:** You must show that the `DatabaseManager` *never* sees the Master Password.
- **Data Isolation:** Show that only the `CryptoEngine` has the methods to handle the `session_key`.
- **Error Handling:** In your diagram, include a "Result" or "Status" return type for your methods (e.g., `+ verify_password() : Boolean`). This shows the system handles failure gracefully.

10.3 Interaction with External Libraries

The most significant interaction is the **`CryptoEngine`** interacting with the **`Cryptography`** **(Fernet/AES)** and **`Hashlib`** libraries.

- **Significance:** This is the "Heart" of the vault. Without these external libraries, the system cannot perform AES-256 or PBKDF2.
- **DatabaseManager** interacts with the **`sqlite3`** library.
- **GUI** interacts with **`Tkinter`** or **`CustomTkinter`**.

10.4 Additional Classes for a Complete Overview

To make your design truly "Complete," you should add two small but vital utility classes:

- **ConfigLoader**: A class that reads the local configuration (like iteration counts or UI themes) from a .json or .ini file.
- **Logger**: A class that writes system events (Login attempts, Errors, Database updates) to a local log file for auditing. *Recruiters love seeing a logging module.*

10.5 Primary Goal of this Diagram

For the project, the primary goal is **System Architecture Documentation & Code Modularity**.